SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	B.Vani	**Reg. No.:**	192472348
Section / Batch:	CSE-AI / 2024	**Date:**	11-08-2026

Code of Conduct

I _______B.VANI_(192472348)_________________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____B.VANI_____________________

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________

HMM-Based POS Tagging

1. Separate words and POS tags

From the training corpus:

Sentence	Words	POS Tags
1	The boy eats rice	DT NN VBZ NN
2	The girl drinks milk	DT NN VBZ NN
3	A cat drinks milk	DT NN VBZ NN
4	The dog chases cat	DT NN VBZ NN
5	A teacher teaches students	DT NN VBZ NNS
6	Students study English	NNS VBP NN
7	Birds fly high	NNS VBP RB
8	Children play games	NNS VBP NNS

The POS tags used are:

- **DT** = Determiner
- **NN** = Singular noun
- **NNS** = Plural noun
- **VBZ** = Verb, 3rd-person singular present
- **VBP** = Verb, non-3rd-person singular present
- **RB** = Adverb

---

2. Emission Probabilities

The emission probability is:

[P(word|tag)=\frac{\text{Count(word, tag)}}{\text{Total count of tag}}]

Tag counts

POS Tag	Total Count
DT	5
NN	10
VBZ	5
NNS	5
VBP	3
RB	1

Emission probabilities

DT

- The occurs 3 times

[P(The|DT)=\frac{3}{5}=0.6]

- A occurs 2 times

[P(A|DT)=\frac{2}{5}=0.4]

NN

There are 10 NN words:

Word	Count	Probability
boy	1	1/10 = 0.1
rice	1	0.1
girl	1	0.1
milk	2	0.2
cat	2	0.2
dog	1	0.1
teacher	1	0.1
English	1	0.1

VBZ

There are 5 VBZ words:

Word	Count	Probability
eats	1	1/5 = 0.2

drinks	2	2/5 = 0.4
chases	1	0.2
teaches	1	0.2

NNS
There are 5 NNS words:

Word	Count	Probability
students	1	0.2
Students	1	0.2
Birds	1	0.2
Children	1	0.2
games	1	0.2

VBP

Word	Count	Probability
study	1	1/3
fly	1	1/3
play	1	1/3

RB

Word	Count	Probability
high	1	1

---

3. Transition Probabilities

The transition probability is:
[P(tag_i|tag_{i-1})

\frac{\text{Count}(tag_{i-1}\rightarrow tag_i)}
{\text{Total transitions from }tag_{i-1}}
]

From the corpus:

DT →

DT always occurs before NN:

[P(NN|DT)=\frac{5}{5}=1]

NN →

NN always occurs before VBZ:

[P(VBZ|NN)=\frac{5}{5}=1]

VBZ →

VBZ is followed by NN 4 times and NNS once.

[P(NN|VBZ)=\frac45=0.8]

[P(NNS|VBZ)=\frac15=0.2]

NNS →

NNS is followed by VBP 3 times:

[P(VBP|NNS)=\frac33=1]

VBP →

VBP has three possible following tags:

[P(NN|VBP)=\frac13]

[P(RB|VBP)=\frac13]

[P(NNS|VBP)=\frac13]

Transition table

From \ To	DT	NN	VBZ	NNS	VBP	RB
DT	0	**1**	0	0	0	0
NN	0	0	**1**	0	0	0
VBZ	0	**0.8**	0	**0.2**	0	0
NNS	0	0	0	0	**1**	0
VBP	0	**1/3**	0	**1/3**	0	**1/3**
RB	0	0	0	0	0	0

---

4. Initial Probabilities

There are 8 training sentences.
The first tags are:

- DT → 5 sentences
- NNS → 3 sentences

Therefore:
[P(DT)=\frac58=0.625]
[P(NNS)=\frac38=0.375]
All other initial probabilities are 0.

5. HMM Using Python Dictionaries

The assessment specifically asks for Python dictionaries, loops and basic programming constructs without built-in NLP/HMM libraries.

```
# Training corpus

corpus = [
    [("The", "DT"), ("boy", "NN"), ("eats", "VBZ"), ("rice", "NN")],
    [("The", "DT"), ("girl", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("A", "DT"), ("cat", "NN"), ("drinks", "VBZ"), ("milk", "NN")],
    [("The", "DT"), ("dog", "NN"), ("chases", "VBZ"), ("cat", "NN")],
    [("A", "DT"), ("teacher", "NN"), ("teaches", "VBZ"), ("students", "NNS")],
    [("Students", "NNS"), ("study", "VBP"), ("English", "NN")],
    [("Birds", "NNS"), ("fly", "VBP"), ("high", "RB")],
    [("Children", "NNS"), ("play", "VBP"), ("games", "NNS")]
]
# Count tags
tag_count = {}
word_tag_count = {}
transition_count = {}
initial_count = {}
```

Calculate counts

```
for sentence in corpus:
    # Initial tag
    first_tag = sentence[0][1]
    if first_tag not in initial_count:
        initial_count[first_tag] = 0

    initial_count[first_tag] += 1
    # Word and tag counts
    for word, tag in sentence:
        if tag not in tag_count:
            tag_count[tag] = 0
        tag_count[tag] += 1
        key = (word, tag)
        if key not in word_tag_count:
            word_tag_count[key] = 0
        word_tag_count[key] += 1
    # Transition counts
    for i in range(len(sentence) - 1):
        current_tag = sentence[i][1]
        next_tag = sentence[i + 1][1]
        key = (current_tag, next_tag)
        if key not in transition_count:
            transition_count[key] = 0
        transition_count[key] += 1
```

6. Calculate Emission and Transition Probabilities

```
emission = {}
for (word, tag), count in word_tag_count.items():

    probability = count / tag_count[tag]
```

```
    emission[(word, tag)] = probability
```

For transitions:

```
transition = {}
for (tag1, tag2), count in transition_count.items():
    # Count total outgoing transitions
    total = 0
    for (a, b), c in transition_count.items():
        if a == tag1:
            total += c
    transition[(tag1, tag2)] = count / total
```

Initial probabilities:

```
initial_probability = {}
total_sentences = len(corpus)
for tag, count in initial_count.items():
    initial_probability[tag] = count / total_sentences
```

---

7. Viterbi Algorithm

The Viterbi algorithm finds the POS-tag sequence having the highest probability.

Formula:

[V_t(tag)=\max_{previous\ tag}
[V_{t-1}(previous\ tag)
\times
P(tag|previous\ tag)
\times
P(word_t|tag)]]

Implementation:

```
def viterbi(sentence, tags):
    viterbi = []
    backpointer = []
    # First word
    first_word = sentence[0]
    first_v = {}
    first_b = {}
    for tag in tags:
        if tag in initial_probability:
            start_prob = initial_probability[tag]
        else:
            start_prob = 0
        emission_prob = emission.get((first_word, tag), 0)
        first_v[tag] = start_prob * emission_prob
        first_b[tag] = None
    viterbi.append(first_v)
    backpointer.append(first_b)
    # Remaining words
    for i in range(1, len(sentence)):
        word = sentence[i]
        current_v = {}
        current_b = {}
        for current_tag in tags:
            emission_prob = emission.get((word, current_tag), 0)
            best_probability = 0
            best_previous_tag = None
            for previous_tag in tags:
```

```
                trans_prob = transition.get(
                    (previous_tag, current_tag), 0
                )
                probability = (
                    viterbi[i - 1][previous_tag]
                    * trans_prob
                    * emission_prob
                )
                if probability > best_probability:
                    best_probability = probability
                    best_previous_tag = previous_tag
            current_v[current_tag] = best_probability
            current_b[current_tag] = best_previous_tag
        viterbi.append(current_v)
        backpointer.append(current_b)
    # Find final best tag
    best_final_tag = None
    best_probability = 0
    for tag in tags:
        if viterbi[-1][tag] > best_probability:
            best_probability = viterbi[-1][tag]
            best_final_tag = tag
    # Backtracking
    best_tags = [best_final_tag]
    for i in range(len(sentence) - 1, 0, -1):
        best_final_tag = backpointer[i][best_final_tag]
        best_tags.append(best_final_tag)
    best_tags.reverse()
    return best_tags, best_probability
```

---

8. Predict POS Tags for "The cat drinks milk"

Input:

The cat drinks milk .Possible sequence from the training corpus:

DT → NN → VBZ → NN

Step 1: The

The is tagged as **DT**.

[P(DT)=0.625]

[P(The|DT)=0.6]

Therefore:

[0.625 \times 0.6 = 0.375]

Step 2: cat

Transition:

[P(NN|DT)=1]

Emission:

[P(cat|NN)=\frac{2}{10}=0.2]

Therefore:

[0.375 \times 1 \times 0.2=0.075]

Step 3: drinks

Transition:

[P(VBZ|NN)=1]

Emission:

[P(drinks|VBZ)=\frac25=0.4]

Therefore:

[0.075\times1\times0.4 =0.03]

Step 4: milk

Transition:

[P(NN|VBZ)=\frac45=0.8]

Emission:

[P(milk|NN)=\frac{2}{10}=0.2]

Therefore:

[0.03\times0.8\times0.2 =0.0048]

So the probability of the sequence is:

[\boxed{0.0048}]

Final Answer

Word	Predicted POS
The	**DT**
cat	**NN**
drinks	**VBZ**
milk	**NN**

Therefore, the predicted POS-tag sequence is: [\boxed{\text{DT NN VBZ NN}}]

The complete HMM path is:

The/DT cat/NN drinks/VBZ milk/NN and its calculated probability is: {0.0048}